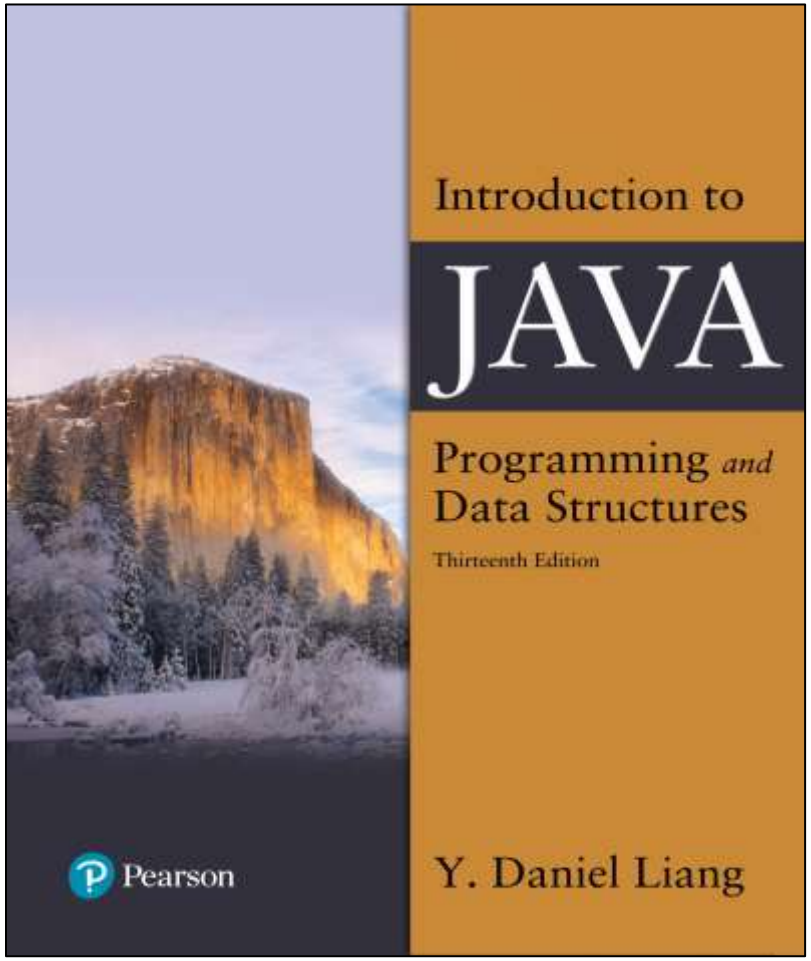
Introduction to Java Programming and Data Structures

Thirteenth Edition



Chapter 4

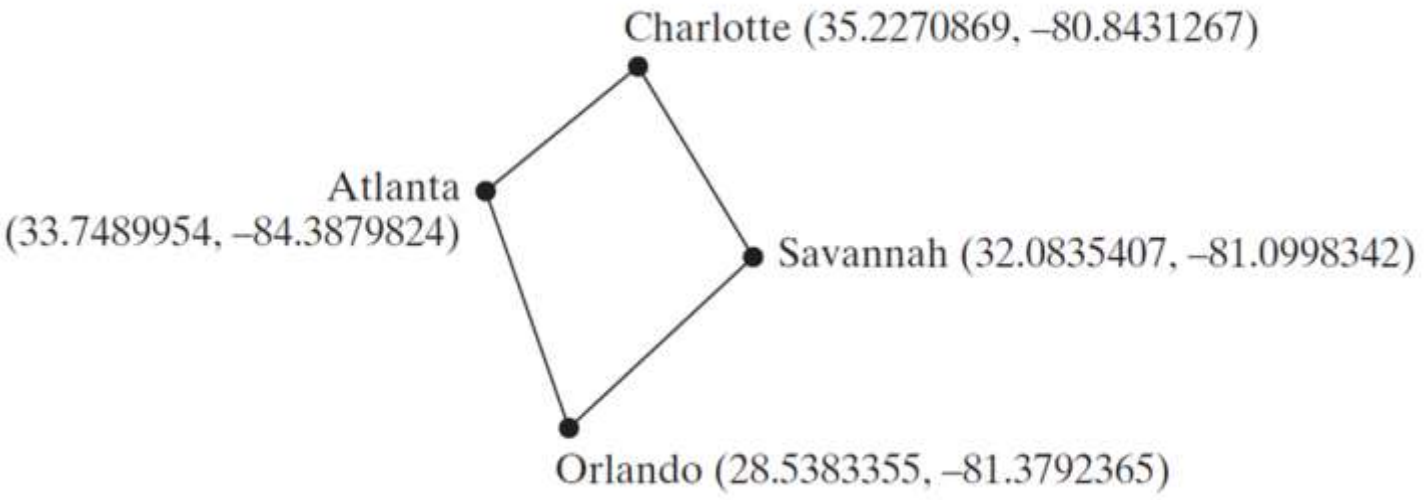
Mathematical Functions, Characters, and Strings

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Motivations

Suppose you need to estimate the area enclosed by four cities, given the GPS locations (latitude and longitude) of these cities, as shown in the following diagram. How would you write a program to solve this problem? You will be able to write such a program after completing this chapter.



Pearson
Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Objectives (1 of 4)

4.1 To solve mathematics problems by using the methods in the **`Math`** class (§4.2).

4.2 To represent characters using the **`char`** type (§4.3).

4.3 To encode characters using ASCII and Unicode (§4.3.1).

4.4 To represent special characters using the escape sequences (§4.4.2).

4.5 To cast a numeric value to a character and cast a character to an integer (§4.3.3).

4.6 To compare and test characters using the static methods in the **`Character`** class (§4.3.4).

4.7 To introduce objects and instance methods (§4.4).

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Objectives (2 of 4)

4.8 To represent strings using the **`String`** objects (§4.4).

4.9 To return the string length using the **`length()`** method (§4.4.1).

4.10 To return a character in the string using the **`charAt(i)`** method (§4.4.2).

4.11 To use the + operator to concatenate strings (§4.4.3).

4.12 To return an uppercase string or a lowercase string and to trim a string (§4.4.4).

4.13 To read strings from the console (§4.4.4).

4.14 To read a character from the console (§4.4.5).

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Objectives (3 of 4)

4.15 To compare strings using the **`equals`** method and the **`compareTo`** methods (§4.4.6).

4.16 To obtain substrings (§4.4.7).

4.17 To find a character or a substring in a string using the **`indexOf`** method (§4.4.8).

4.18 To program using characters and strings (**`GuessBirthday`**) (§4.5.1).

4.19 To convert a hexadecimal character to a decimal value (**`HexDigit2Dec`**) (§4.5.2).

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Objectives (4 of 4)

4.20 To revise the lottery program using strings (**`LotteryUsingStrings`**) (§4.5.3).

4.21 To format output using the **`System.out.printf`** method (§4.6).

4.22 To form multi-line strings using text blocks (§4.7).

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Mathematical Functions

Java provides many useful methods in the **`Math`** class for performing common mathematical functions.

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

The `Math` Class

- Class constants:
  - `PI`
  - `E`
- Class methods:
  - Trigonometric Methods
  - Exponent Methods
  - Rounding Methods
  - `min`, `max`, `abs`, and `random` Methods

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Trigonometric Methods

- `sin(double a)`
- `cos(double a)`
- `tan(double a)`
- `acos(double a)`
- `asin(double a)`
- `atan(double a)`

Radians

toRadians(90)

Examples:

```
Math.sin(0) returns 0.0
Math.sin(Math.PI / 6)
returns 0.5
Math.sin(Math.PI / 2)
returns 1.0
Math.cos(0) returns 1.0
Math.cos(Math.PI / 6)
returns 0.866
Math.cos(Math.PI / 2)
returns 0
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Exponent Methods

- **`exp(double a)`**
  Returns `e` raised to the power of `a`.
- **`log(double a)`**
  Returns the natural logarithm of `a`.
- **`log10(double a)`**
  Returns the 10-based logarithm of `a`.
- **`pow(double a, double b)`**
  Returns `a` raised to the power of `b`.
- **`sqrt(double a)`**
  Returns the square root of `a`.

Examples:

```
Math.exp(1) returns 2.71
Math.log(2.71) returns 1.0
Math.pow(2, 3) returns 8.0
Math.pow(3, 2) returns 9.0
Math.pow(3.5, 2.5) returns 22.91765
Math.sqrt(4) returns 2.0
Math.sqrt(10.5) returns 3.24
```

Pearson
Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Rounding Methods

- **`double ceil(double x)`**

  `x` rounded up to its nearest integer. This integer is returned as a double value.
- **`double floor(double x)`**

  `x` is rounded down to its nearest integer. This integer is returned as a double value.
- **`double rint(double x)`**

  `x` is rounded to its nearest integer. If `x` is equally close to two integers, the even one is returned as a double.
- **`int round(float x)`**

  `Return (int)Math.floor(x+0.5).`
- **`long round(double x)`**

  `Return (long)Math.floor(x+0.5).`

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Rounding Methods Examples (1 of 2)

```
Math.ceil(2.1) returns 3.0
Math.ceil(2.0) returns 2.0
Math.ceil(-2.0) returns -2.0
Math.ceil(-2.1) returns -2.0
Math.floor(2.1) returns 2.0
Math.floor(2.0) returns 2.0
Math.floor(-2.0) returns -2.0
Math.floor(-2.1) returns -3.0
Math.rint(2.1) returns 2.0
Math.rint(2.0) returns 2.0
Math.rint(-2.0) returns -2.0
Math.rint(-2.1) returns -2.0
Math.rint(2.5) returns 2.0
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Rounding Methods Examples (2 of 2)

```
Math.rint(-2.5) returns -2.0
Math.round(2.6f) returns 3
Math.round(2.0) returns 2
Math.round(-2.0f) returns -2
Math.round(-2.6) returns -3
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

`min`, `max`, and `abs`

- `max(a, b)` and `min(a, b)`

  Returns the maximum or minimum of two parameters.
- `abs(a)`

  Returns the absolute value of the parameter.
- `random()`

  Returns a random double value in the range [0.0, 1.0).

Examples:

```
Math.max(2, 3) returns 3

Math.max(2.5, 3) returns
   3.0

Math.min(2.5, 3.6)
   returns 2.5

Math.abs(-2) returns 2

Math.abs(-2.1) returns
   2.1
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

The `random` Method

Generates a random **`double`** value greater than or equal to 0.0 and less than 1.0 (**`0 <= Math.random() < 1.0`**).

Examples:

```
(int)(Math.random() * 10)
```
→ Returns a random integer between 0 and 9.

```
50 + (int)(Math.random() * 50)
```
→ Returns a random integer between 50 and 99.

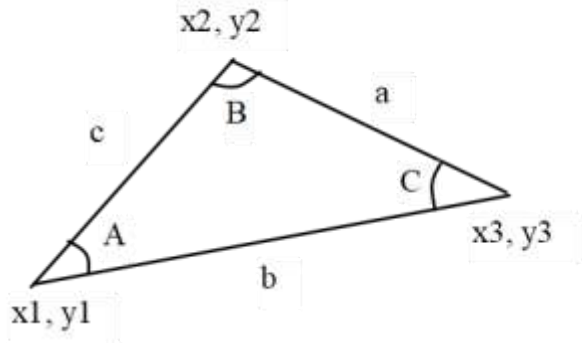
In general,

```
a + Math.random() * b
```
→ Returns a random number between a and a + b, excluding a + b.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Case Study: Computing Angles of a Triangle




```
A = acos((a * a - b * b - c * c) / (-2 * b * c))
B = acos((b * b - a * a - c * c) / (-2 * a * c))
C = acos((c * c - b * b - a * a) / (-2 * a * b))
```

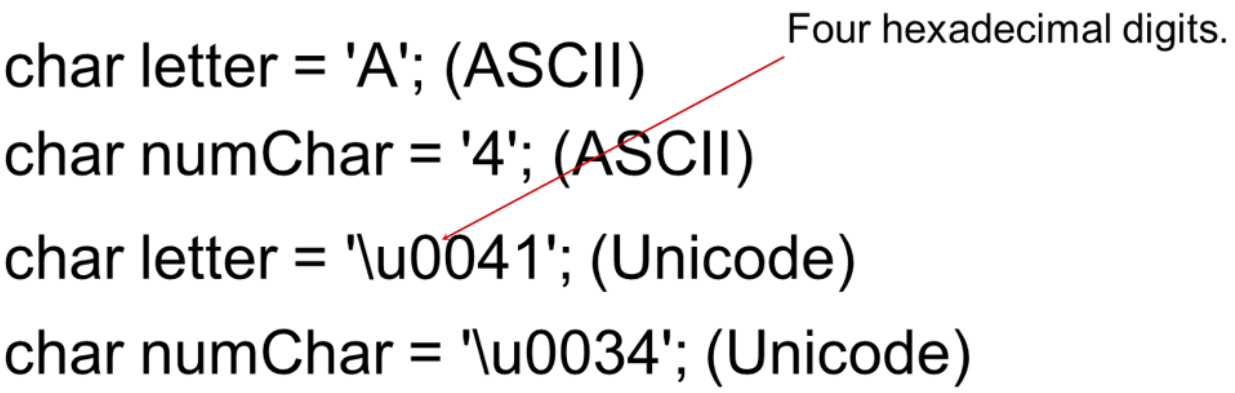
Write a program that prompts the user to enter the x- and y-coordinates of the three corner points in a triangle and then displays the triangle's angles.

ComputeAngles

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

`Character` Data Type



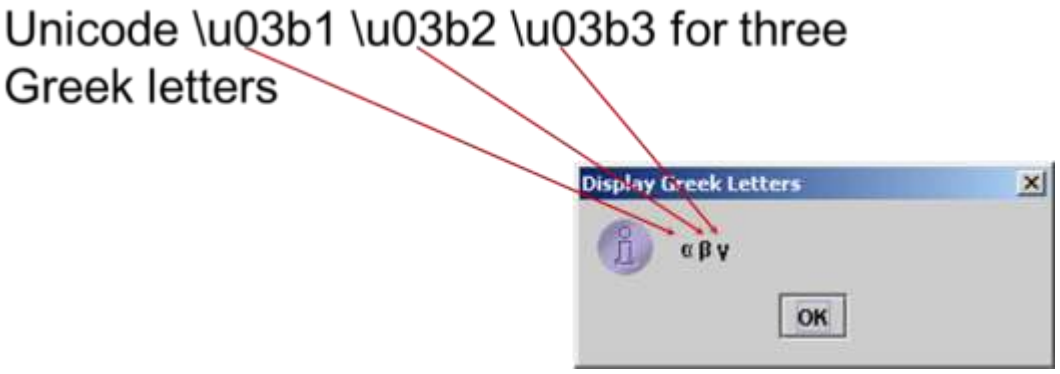
Note: The increment and decrement operators can also be used on **`char`** variables to get the next or preceding Unicode character. For example, the following statements display character **b**.

```
char ch = 'a';
System.out.println(++ch);
```

Pearson
Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Unicode Format

Java characters use **Unicode**, a 16-bit encoding scheme established by the Unicode Consortium to support the interchange, processing, and display of written texts in the world's diverse languages. Unicode takes two bytes, preceded by \u, expressed in four hexadecimal numbers that run from **'\u0000'** to **'\uFFFF'**. So, Unicode can represent `65535 + 1 characters`.



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

ASCII Code for Commonly Used Characters

Characters	Code Value in Decimal	Unicode Value
'0' to **'9'**	48 to 57	\u0030 to \u0039
'A' to **'Z'**	65 to 90	\u0041 to \u005A
'a' to **'z'**	97 to 122	\u0061 to \u007A

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Escape Sequences for Special Characters

Escape Sequence	Name	Unicode Code	Decimal Value
\b	Backspace	**\u0008**	8
\t	Tab	**\u0009**	9
\n	Linefeed	**\u000A**	10
\f	Formfeed	**\u000C**	12
\r	Carriage Return	**\u000D**	13
****	Backslash	**\u005C**	92
\”	Double Quote	**\u0022**	34

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Appendix B: ASCII Character Set (1 of 2)

ASCII Character Set is a subset of the Unicode from \u0000 to \u007f

Table B.1 ASCII Character Set in the Decimal Index

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
0	nul	soh	stx	etx	eot	enq	ack	bel	bs	ht
1	nl	vt	ff	cr	so	si	dle	dcl	dc2	dc3
2	dc4	nak	syn	etb	can	em	sub	esc	fs	gs
3	rs	us	sp	!	“	#	$	%	&	‘
4	(	)	*	+	,	-	.	/	0	1
5	2	3	4	5	6	7	8	9	:	;
6	<	=	>	?	@	A	B	C	D	E
7	F	G	H	I	J	K	L	M	N	O
8	P	Q	R	S	T	U	V	W	X	Y
9	Z	[	\	]	^	_	`	a	b	c
10	d	e	f	g	h	i	j	k	l	m
11	n	o	p	q	r	s	t	u	v	w
12	x	y	z	{	|	}	~	del		

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Appendix B: ASCII Character Set (2 of 2)

ASCII Character Set is a subset of the Unicode from \u0000 to \u007f

Table B.2 ASCII Character Set in the Hexadecimal Index

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**A**	**B**	**C**	**D**	**E**	**F**
0	nul	soh	stx	etx	eot	enq	ack	bel	bs	ht	nl	vt	ff	cr	so	si
1	dle	dcl	dc2	dc3	dc4	nak	syn	etb	can	em	sub	esc	fs	gs	rs	us
2	sp	!	“	#	$	%	&	‘	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	‘	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	del

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Casting Between `char` and Numeric Types

```
int i = 'a'; // Same as int i = (int)'a';

char c = 97; // Same as char c = (char)97;
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Comparing and Testing Characters

```
if (ch >= 'A' && ch <= 'Z')
  System.out.println(ch + " is an uppercase letter");
else if (ch >= 'a' && ch <= 'z')
  System.out.println(ch + " is a lowercase letter");
else if (ch >= '0' && ch <= '9')
  System.out.println(ch + " is a numeric character");
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Methods in the `Character` Class

Method	Description
`isDigit(ch)`	Returns true if the specified character is a digit.
`isLetter(ch)`	Returns true if the specified character is a letter.
`isLetterOfDigit(ch)`	Returns true if the specified character is a letter or digit.
`isLowerCase(ch)`	Returns true if the specified character is a lowercase letter.
`isUpperCase(ch)`	Returns true if the specified character is an uppercase letter.
`toLowerCase(ch)`	Returns the lowercase of the specified character.
`toUpperCase(ch)`	Returns the uppercase of the specified character.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

The `String` Type

The char type only represents one character. To represent a string of characters, use the data type called String. For example,

```
String message = "Welcome to Java";
```

String is actually a predefined class in the Java library just like the System class and Scanner class. The String type is not a primitive type. It is known as a **reference type**. Any Java class can be used as a reference type for a variable. Reference data types will be thoroughly discussed in Chapter 9, “Objects and Classes.” For the time being, you just need to know how to declare a String variable, how to assign a string to the variable, how to concatenate strings, and to perform simple operations for strings.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Simple Methods for `String` Objects (1 of 2)

Method	Description
`length()`	Returns the number of characters in this string.
`charAt(index)`	Returns the character at the specified index from this string.
`concat(s1)`	Returns a new string that concatenates this string with string `s1`.
`toUpperCase()`	Returns a new string with all letters in uppercase.
`toLowerCase()`	Returns a new string with all letters in lowercase.
`trim()`	Returns a new string with whitespace characters trimmed on both sides.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Simple Methods for `String` Objects (2 of 2)

Strings are objects in Java. The methods in the preceding table can only be invoked from a specific string instance. For this reason, these methods are called **instance methods**. A non-instance method is called a **static method**. A static method can be invoked without using an object. All the methods defined in the **`Math`** class are static methods. They are not tied to a specific object instance. The syntax to invoke an instance method is

`referenceVariable.methodName(arguments)`.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Getting `String` Length

```
String message = "Welcome to Java";

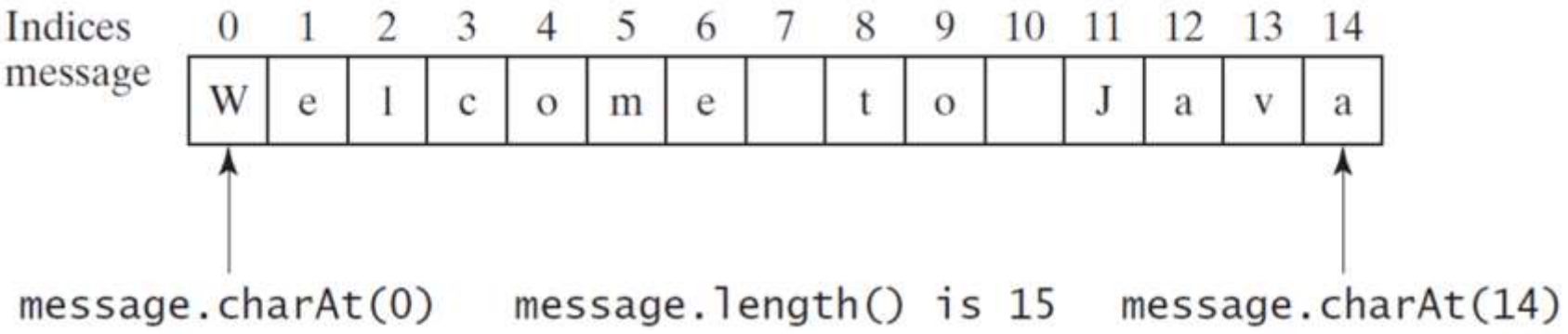
System.out.println("The length of " + message + " is "

 + message.length());
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Getting Characters From a `String`




```
String message = "Welcome to Java";

System.out.println("The first character in message is "
 + message.charAt(0));
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Converting Strings

`"Welcome".toLowerCase()` returns a new string, welcome.

`"Welcome".toUpperCase()` returns a new string, WELCOME.

`" Welcome ".trim()` returns a new string, Welcome.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

`String` Concatenation

```
String s3 = s1.concat(s2); or String s3 = s1 + s2;

// Three strings are concatenated
String message = "Welcome " + "to " + "Java";

// String Chapter is concatenated with number 2
String s = "Chapter" + 2; // s becomes Chapter2

// String Supplement is concatenated with character B
String s1 = "Supplement" + 'B'; // s1 becomes SupplementB
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Reading a `String` From the Console

```
Scanner input = new Scanner(System.in);

System.out.print("Enter three words
separated by spaces: ");

String s1 = input.next();

String s2 = input.next();

String s3 = input.next();

System.out.println("s1 is " + s1);

System.out.println("s2 is " + s2);

System.out.println("s3 is " + s3);
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Reading a `Character` From the Console

```
Scanner input = new Scanner(System.in);

System.out.print("Enter a character: ");

String s = input.nextLine();

char ch = s.charAt(0);

System.out.println("The character entered is " + ch);
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Comparing Strings

Method	Description
`equals(s1)`	Returns true if this string is equal to string `s1`.
`equalsIgnoreCase(s1)`	Returns true if this string is equal to string `s1`; it is case insensitive.
`compareTo(s1)`	Returns an integer greater than `0`, equal to `0`, or less than `0` to indicate whether this string is greater than, equal to, or less than `s1`.
`compareToIgnoreCase(s1)`	Same as `compareTo` except that the comparison is case insensitive.
`startsWith(prefix)`	Returns true if this string starts with the specified prefix.
`endsWith(suffix)`	Returns true if this string ends with the specified suffix.

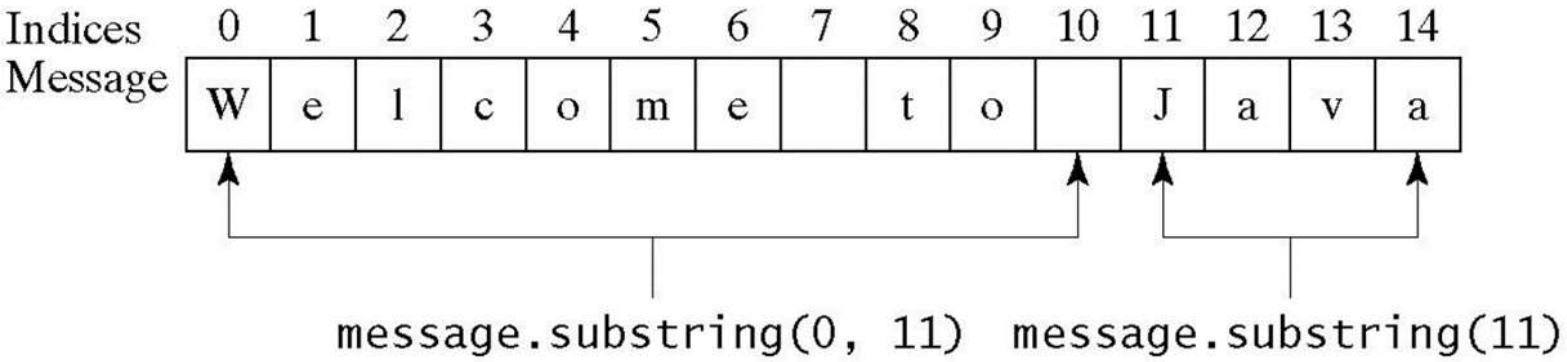
OrderTwoCities

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Obtaining Substrings

Method	Description
`substring(beginIndex)`	Returns this string's substring that begins with the character at the specified `beginIndex` and extends to the end of the string, as shown in Figure 4.2.
`substring(beginIndex, endIndex)`	Returns this string's substring that begins at the specified `beginIndex` and extends to the character at index `endIndex - 1`, as shown in Figure 9.6. Note that the character at `endIndex` is not part of the substring.



Pearson
Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Finding a `Character` or a `Substring` in a `String` (1 of 2)

Method	Description
`indexOf(ch)`	Returns the index of the first occurrence of `ch` in the string. Returns `-1` if not matched.
`indexOf(ch, fromIndex)`	Returns the index of the first occurrence of `ch` after `fromIndex` in the string. Returns `-1` if not matched.
`indexOf(s)`	Returns the index of the first occurrence of string `s` in this string. Returns `-1` if not matched.
`indexOf(s, fromIndex)`	Returns the index of the first occurrence of string `s` in this string after `fromIndex`. Returns `-1` if not matched.
`lastIndexOf(ch)`	Returns the index of the last occurrence of `ch` in the string. Returns `-1` if not matched.
`lastIndexOf(ch, fromIndex)`	Returns the index of the last occurrence of `ch` before `fromIndex` in this string. Returns `-1` if not matched.
`lastIndexOf(s)`	Returns the index of the last occurrence of string `s`. Returns `-1` if not matched.
`lastIndexOf(s, fromIndex)`	Returns the index of the last occurrence of string `s` before `fromIndex`. Returns `-1` if not matched.

Pearson

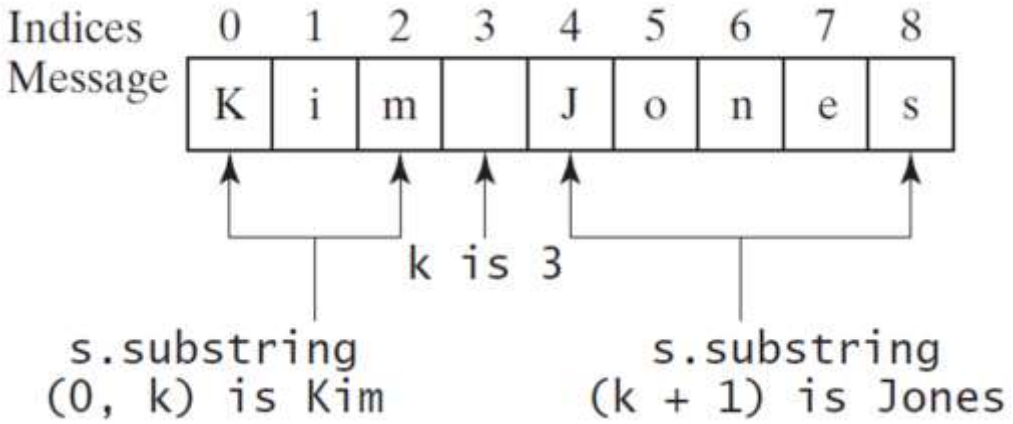
Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Finding a `Character` or a `Substring` in a `String` (2 of 2)

```
int k = s.indexOf(' ');

String firstName = s.substring(0, k);

String lastName = s.substring(k + 1);
```



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Conversion Between Strings and Numbers

```
int intValue = Integer.parseInt(intString);

double doubleValue = Double.parseDouble(doubleString);

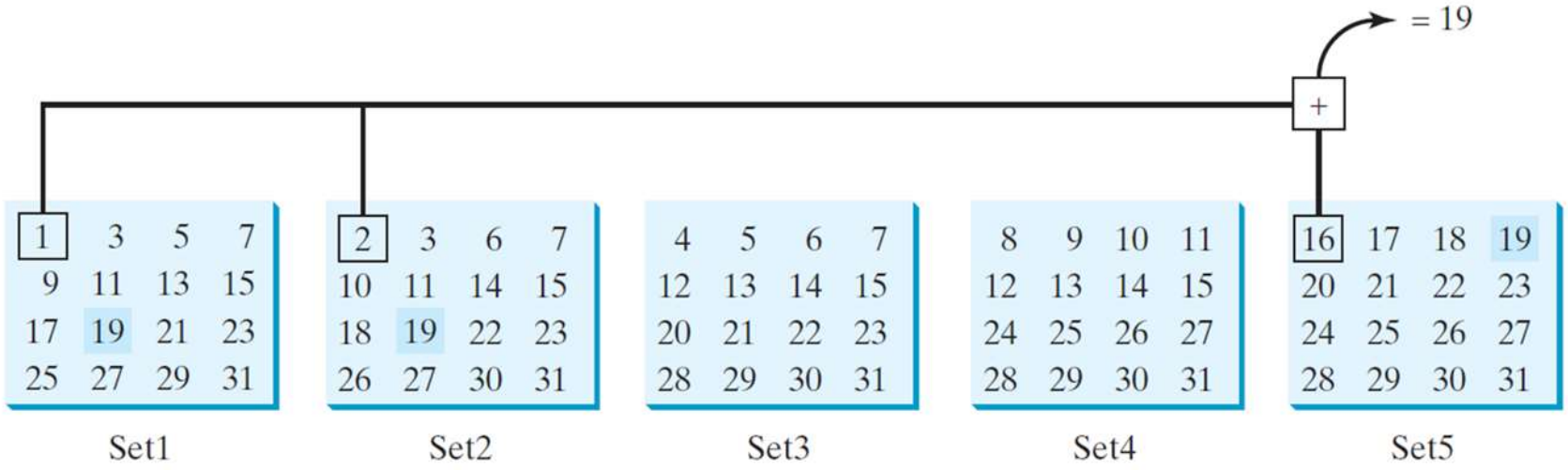

String s = number + "";
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Problem: Guessing Birthday

The program can guess your birth date. Run to see how it works.



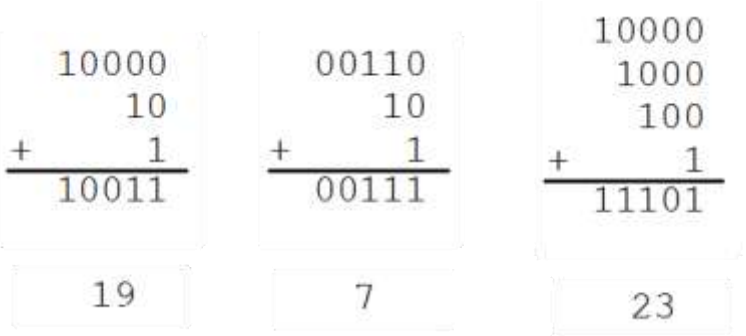
GuessBirthday

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Mathematics Basis for the Game

19 is 10011 in binary. 7 is 111 in binary. 23 is 11101 in binary



Decimal	Binary
1	00001
2	00010
3	00011
...	
19	10011
...	
31	11111

$$\begin{array}{r} b_5\ 0\ 0\ 0\ 0 \\ b_4\ 0\ 0\ 0 \\ b_3\ 0\ 0 \\ b_2\ 0 \\ +\quad\quad b_1 \\ \hline b_5\, b_4\, b_3\, b_2\, b_1 \end{array} \qquad \begin{array}{r} 10000 \\ 10 \\ +\quad 1 \\ \hline 10011 \end{array} \qquad \begin{array}{r} 10000 \\ 1000 \\ 100 \\ 10 \\ +\quad 1 \\ \hline 11111 \end{array}$$

19 31

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Case Study: Converting a Hexadecimal Digit to a Decimal Value

Write a program that converts a hexadecimal digit into a decimal value.

HexDigit2Dec

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Case Study: Revising the Lottery Program Using Strings

A problem can be solved using many different approaches. This section rewrites the lottery program in Listing 3.7 using strings. Using strings simplifies this program.

LotteryUsingStrings

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Formatting Output

Use the `printf` statement.

```
System.out.printf(format, items);
```

Where format is a string that may consist of substrings and format specifiers. A format specifier specifies how an item should be displayed. An item may be a numeric value, character, boolean value, or a string. Each specifier begins with a percent sign.

Pearson

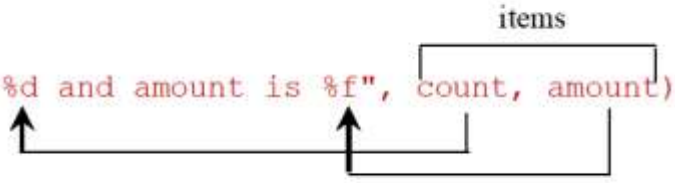
Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Frequently-Used Specifiers

Specifier	Output	Example
%b	a boolean value	true or false
%c	a character	'a'
%d	a decimal integer	200
%f	a floating-point number	45.460000
%e	a number in standard scientific notation	4.556000e+01
%s	a string	"Java is cool"

```
int count = 5;
double amount = 45.56;
System.out.printf("count is %d and amount is %f", count, amount);

display          count is 5 and amount is 45.560000
```



Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

FormatDemo

The example gives a program that uses **`printf`** to display a table.

FormatDemo

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Text Blocks

Text blocks became a standard feature since Java 15. It enables you to form a multi-line string. Here is an example of a text block:

```
String tb = """
            Java
            Python
            C++""";
System.out.println(tb);
```

The code displays three lines

```
Java
Python
C++
```

A text block contains multiple line. The first line starts with three double quotation marks (""") followed by space characters. The last line ends with three double quotation marks (""").

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Indentation in Text Blocks

You can use indentation in a text block. To achieve this, the compiler first removes the maximum left common space characters for all lines. For example, the maximum common left space characters for all lines in the following text block is 2, because there are two leading space characters before "Java".

```
String tb = """
  Java
    Python
      C++""";
```

After removing two leading space characters from each line, the resulting string is

```
Java
  Python
    C++
```

After removing the maximum common left space characters, the indentation of the text block is preserved.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Indentation in Text Blocks: ending """

Note if the ending """ is on a separate line by itself, its leading spaces are counted in the maximum common left space. For example, in the following code,

```
String tb = """
   Java
     Python
       C++
""";
```

The maximum common left space is 0, because there are no space before the ending """. The resulting string will be

```
Java
    Python
      C++
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Right Trailing Space

The right trailing space are trimmed by default. If you want to keep right trailing space, use `\s`. For example, the following code keep three right space characters after "Python".

```
String tb = """
   Java
     Python  \s
       C++""";
```

Note that there are two spaces after `Python` and before `\s`. Including `\s`, there will be three spaces after "Python".

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Escape Characters

You used `\s` escape character for a space. You can also use other escape characters in needed. For example, for the following text block:

```
String tb = """
  Java
  \tPython\n
  C++""";
```

The resulting string is

```
Java
        Python

C++
```

The space before "`Python`" is due to the tab character (`\t`). The space line after "`Python`" is due to the new line character "`\n`".

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

""" in Text Block as Literal

If you need to put """ into the text block, you can use \""". For example, for the following text block:

```
String tb = """
  Java\"""
  Python
  C++""";
```

The resulting string is

```
Java"""
Python
C++
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

New `String` Methods

As part of the new feature on text blocks, several new String methods are introduced. You can use the `stripIndent()` to remove the maximum common left space from a multi-line string in the same way as a text block is processed.

You can use the `formatted(args)` method to format a `string`. For example, for the following code:

```
String tb = """
  Product: %s
  Price: $%.2f""".formatted("Salt", 4.52);
```

The resulting string is

```
Produce: Salt
Price: $4.52
```

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved

Copyright



This work is protected by United States copyright laws and is provided solely for the use of instructors in teaching their courses and assessing student learning. Dissemination or sale of any part of this work (including on the World Wide Web) will destroy the integrity of the work and is not permitted. The work and materials from it should never be made available to students except by instructors using the accompanying text in their classes. All recipients of this work are expected to abide by these restrictions and to honor the intended pedagogical purposes and the needs of other instructors who rely on these materials.

Pearson

Copyright © 2024 Pearson Education, Inc. All Rights Reserved